GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization

- Background: multi-reward GRPO
  - Normally, the reward of GRPO is from a single perspective. For example, correctness, "either TRUE (1) or FALSE (0)".
  - In multi-reward RL, for each rollout in each task, we have multiple reward models to evaluate the answer from multiple perspectives. For example, correctness (binary), helpfulness (0-10), and precision (0-10).

> This is a classic method in robotics, but it has not been introduced to LLM until very recently.

  - In multi-reward GRPO, for each rollout in a group, we add all rewards up and continue the routine of GRPO. Formally:

    $r_{\text{sum}}^{(i,j)} = r_1^{(i,j)} + \cdots + r_n^{(i,j)}$

    $A_{\text{sum}}^{(i,j)} = \frac{r_{\text{sum}}^{(i,j)} - \text{mean}\{r_{\text{sum}}^{(i,1)}, \ldots, r_{\text{sum}}^{(i,G)}\}}{\text{std}\{r_{\text{sum}}^{(i,1)}, \ldots, r_{\text{sum}}^{(i,G)}\}}$

    where:

    $$\begin{aligned} i &\text{ is the i-th task} \\ G &\text{ is the group size} \\ j &\text{ is the j-th rollout} \\ n &\text{ is the \# of perspectives of rewards} \end{aligned}$$

  - However, this type of GRPO has a significant drawback: it inherently *compresses* the reward signal, causing loss of information in the advantage estimates.

> To illustrate this, consider the following example:

  - Why does this happen? When two rollouts have a huge difference (0 & 2), their stdev is also huge. The division in normalization will eliminate the difference, which is exactly what we want to be shown in advantages.

- It is not a hard process to solve this problem. We just need to swap the order of calculation. **Instead of add->normalize (advantage), we normalize -> add.**
  - Firstly normalize each perspective of rewards for each rollout (intragroup)

    $A_1^{(i,j)} = \frac{r_1^{(i,j)} - \text{mean}\{r_1^{(i,1)}, \ldots, r_1^{(i,G)}\}}{\text{std}\{r_1^{(i,1)}, \ldots, r_1^{(i,G)}\}}, \quad \ldots, \quad A_n^{(i,j)} = \frac{r_n^{(i,j)} - \text{mean}\{r_n^{(i,1)}, \ldots, r_n^{(i,G)}\}}{\text{std}\{r_n^{(i,1)}, \ldots, r_n^{(i,G)}\}}$

    where:

    $$\begin{aligned} i &\text{ is the i-th task} \\ G &\text{ is the group size} \\ j &\text{ is the j-th rollout} \\ n &\text{ is the \# of perspectives of rewards} \end{aligned}$$

  - Then, add up the normalized rewards:

    $A_{\text{sum}}^{(i,j)} = A_1^{(i,j)} + \cdots + A_n^{(i,j)}$

- As there can be multiple perspectives, the total reward can be very large (100 perspectives, each got 0.7, then in total 70). To ensure stability, do the batch-norm afterwards.

  $$\hat{A}_{\text{sum}}^{(i,j)} = \frac{A_{\text{sum}}^{(i,j)} - \text{mean}\{A_{\text{sum}}^{(i',j')} | i' \in D_{\text{Batch}}, j'=1,\ldots,G\}}{\text{std}\{A_{\text{sum}}^{(i',j')} | i' \in D_{\text{Batch}}, j'=1,\ldots,G\} + \epsilon}$$

- By norm-then-add, the *differences* in each perspective are naturally kept during addition. The batch norm does not eliminate the *differences* either, as it applies a global linear transformation (scaling) to the entire batch, which strictly preserves the relative order and magnitude ratios between different samples (e.g., the aggregated score of 1.414 will still remain exactly 2x larger than 0.707 after global scaling).

- Miscellaneous optimization
  - Practically yet classically , when we add the perspective of a reward up, we give each perspective a **frozen** weight:

    $$A_{\text{sum}}^{(i,j)} = w_1 A_1^{(i,j)} + \cdots + w_n A_n^{(i,j)}$$

  - However, the simple (or weighted, as shown above) addition ignores a crucial problem: some perspectives are hard (e.g., *"CoT is completely correct"*), while others are easy (e.g., *"make sure the output is understandable by at least one human"*). The model, in this case, will tend to fulfill the easiest perspective, causing a bias in task handling.
  - Solution: make the hard perspective a **prerequisite** to some easy perspectives. That is to say, unless you achieve a threshold score of $t$in the hard perspectives, you cannot gain the reward of those easy perspectives.

    $$r_k = \begin{cases} r_k, & \text{if } r_l \geq t \\ 0, & \text{otherwise.} \end{cases}$$

    $$\text{where}$$

    $$r_k \text{ is the easy perspective's reward}$$

    $$r_l \text{ is the hard perspective's reward}$$

- Experiments
  - Experimental Setup
    - Models: Qwen2.5-Instruct (1.5B, 3B), DeepSeek-R1 (1.5B, 7B), and Qwen3-4B-Instruct.
    - Baselines: Standard GRPO and GRPO w/o std (GRPO without standard deviation normalization).
    - Framework: Implementations based on verl.
  - Task 1: Tool Calling (Accuracy vs. Format)
    - Objective: Optimize for correct tool usage (Correctness Reward) and strict output structure (Format Reward).
    - Dataset: ToolRL training set (ToolACE, Hammar, xLAM); Evaluated on Berkeley Function Call Leaderboard (BFCL-v3).
    - Results:
      - GDPO consistently outperformed GRPO in both accuracy and format compliance.
      - For Qwen2.5-1.5B, GDPO achieved ~2.7% higher overall accuracy and >4% higher format correctness compared to GRPO.
      - Ablation Study (GRPO w/o std):
        - Removing the standard deviation term from GRPO improved correctness

slightly but resulted in a 0% correct format ratio.
        - Conclusion: Simple tweaks to GRPO cause instability; GDPO's decoupled normalization is necessary for multi-objective convergence.
    - Task 2: Mathematical Reasoning (Accuracy vs. Length)
        - Objective: Improve accuracy while enforcing a length constraint (implicitly competing rewards).
        - Observation (The "Collapse"):
            - GRPO: Initially learned the length constraint but suffered from "reward collapse" after ~400 steps, where accuracy dropped and response length exploded.
            - GDPO: Maintained stable length control while continuously improving accuracy throughout training.
            - Conditioned Rewards (The Trick):
                - When using Conditioned Rewards (length reward is only given if the answer is correct), GDPO effectively prioritized accuracy.
                - GDPO achieved a 4.4% accuracy gain on AIME with 16.9% fewer length violations compared to GRPO.
    - Task 3: Coding Reasoning (Scalability to 3 Rewards)
        - Objective: Optimize three rewards simultaneously: Pass Rate (Correctness), Length, and Bug Ratio.
        - Dataset: DeepSeek-R1-7B trained on Eurus-2-RL.
        - Results:
            - GDPO proved effective even with increased reward dimensions ($GDPO_3 - obj$).
            - Compared to GRPO, GDPO maintained similar pass rates but significantly reduced the bug ratio and length violations.
    - Key Takeaways
        - Stability: GDPO prevents the "training collapse" often seen in GRPO during later stages of multi-reward training.
        - Resolution: GDPO preserves the relative difference between "good" and "better" responses (e.g., meeting 1 vs. 2 objectives), whereas GRPO tends to flatten these signals.
        - Pareto Frontier: Across all tasks, GDPO offers a better trade-off between conflicting objectives (e.g., accuracy vs. efficiency).
- Work Cited
    - Paper
    - Chat History